Ethereum

Solidity

• **Bitcoin is a decentralized application for money.**
• **Ethereum is a decentralized platform for *any* application.**

With **Bitcoin**, you can only write down debit and credit transactions: "Alice pays Bob 1 BTC."

- With **Ethereum**, you can write down debit/credit transactions, but you can *also* write down a set of rules in the notebook itself. For example: "If Alice pays Bob 1 ETH, and today is a Tuesday, then automatically send 0.1 ETH from Bob to Carol."

Ethereum Virtual Machine (EVM)

How can thousands of different computers, all over the world, run the same piece of code and be **100% certain** they all get the exact same result?

Think about your own computer. The result of a program can depend on the time of day, a random number, a file on your hard drive, or information you fetch from the internet. If every computer on the Ethereum network tried to run a program that said "fetch the price of gold from google.com," they might all get slightly different answers at slightly different times. The network would instantly fall out of consensus. The ledger would break.

The system needs to be perfectly **deterministic**. For a given input, the output must *always* be the same, no matter who runs it or when they run it.

Ethereum solves this problem with a brilliant concept: the **Ethereum Virtual Machine (EVM)**.

- **It's a Specification, Not a Physical Thing:** The EVM is not a piece of hardware. It's a detailed blueprint for a CPU. It's a set of rules. Anyone can use this blueprint to build their own software version of the EVM (and they do, in different programming languages like Go, Rust, and Java). As long as everyone's software version follows the rules exactly, they are guaranteed to produce the same output.
- **It's Completely Isolated (Sandboxed):** Code running inside the EVM has no access to the outside world. It cannot access the computer's network, its file system, or other processes. It lives in a "walled garden." The only information it can "see" is information that is already on the blockchain itself (like the current block number, account balances, or data stored by other contracts). This is the key to achieving determinism.
- **It has a Simple Instruction Set:** Just like a real CPU has basic instructions like ADD, COPY, STORE, the EVM has its own set of simple instructions called **opcodes**. When a developer writes a program in a high-level language like Solidity (Ethereum's most popular language), it gets compiled down into these simple, universal EVM opcodes. This is what is actually stored on the blockchain.

No Randomness,No External API Calls and No Floating-Point Math

GAS Fees:

- **Programmer A** writes very clean, efficient code to calculate the average of 10 numbers.
- **Programmer B** writes messy, inefficient code to do the same task. It has unnecessary steps and loops.

Imagine the user of that program is a bit optimistic and thinks the code is more efficient than it is. They set the **Gas Limit** on their transaction to **50,000**. What happens to their transaction, and what happens to the fee they paid?

Two Types of Account in Ethereum

1. Externally Owned Accounts (EOAs)

- **What it is:** This is the account you and I would create with a wallet like MetaMask or Ledger.

- **How it's controlled:** It is controlled by a private key. Only the person with the private key can sign and authorize transactions from this account.
- **What it can do:** It can hold and send Ether (ETH). It can also initiate transactions to interact with other accounts (both EOAs and contracts).
- **Key Property:** An EOA **cannot** have code associated with it. It is passive. It only reacts when its owner tells it to do something.

2. Contract Accounts

- **What it is:** This is an account that is created when a developer deploys a smart contract to the network.
- **How it's controlled:** It is controlled by its own internal **code**. It has no private key. It "wakes up" and executes its code when it receives a transaction from an EOA or a message from another contract.
- **What it can do:** It can hold and send ETH. It can do anything its code tells it to do: run complex calculations, read/write its own storage, and even call other contract accounts.
- **Key Property:** A Contract Account is an autonomous agent living on the blockchain. It's a robot waiting for a command. This is where the code actually lives.

Key part of account

- **Nonce:** A counter that goes up by one every time the account *sends* a transaction. This is a security feature to prevent someone from taking one of your old transactions and "replaying" it to send money again.
- **Balance:** The amount of Ether (the native currency) this account owns, stored as a simple number.
- **codeHash:** If this row represents a smart contract, this column contains the hash of its code. If it's a regular user account (an EOA), this column is empty. This is how the network distinguishes between users and code.
- **storageRoot:** If this is a smart contract, it has its own little private database or "storage." This column contains the root hash of that storage, which is a fingerprint of all the data the contract is holding.

How to send Transaction

- `from: Your Address`
- `to: Your Friend's Address`
- `value: 1 ETH`
- `gasLimit: 21,000 (standard for a simple transfer)`
- `gasPrice: 20 Gwei`
- `nonce: 74 (your account's next transaction number, fetched from the network)`

Where the transaction is stored and where the balance is stored?

The **Ethereum state** refers to the current status of **all accounts and smart contracts** on the Ethereum blockchain at a specific point in time.

```
Old State + Transactions = New State
```

The "accounts" are not stored inside the blocks. They are stored in a separate, massive database on each node's hard drive. The block only contains a cryptographic *proof* (the stateRoot) that this database was updated correctly.

Database 1: The Blockchain (The Journal)

- **What it is:** A simple, linear list of blocks.
- **How it's stored:** This is literally a set of files on your hard drive, one after another (block_0, block_1, block_2, ...). Each block file contains its header and the list of all transactions included in it.
- **How it grows:** It only ever grows. You just keep appending new blocks to the end of the chain. It's an append-only log. This is relatively simple data storage.

Database 2: The State (The Current Balance Sheet)

- **What it is:** The massive "spreadsheet" of all accounts and their properties (nonce, balance, storage, code). This is a much more complex data structure.

- **How it's stored:** This is stored in a highly optimized key-value database on your node's hard drive (commonly using a database like LevelDB). The "key" is the account address, and the "value" is the account's data. This database is structured as a **Merkle Patricia Trie**.
    It's a special kind of database that has one magical property: you can generate a single, 32-byte hash (the stateRoot) that acts as a unique fingerprint for the *entire contents of the database*. If even one bit changes in any account's balance anywhere, the final stateRoot hash will change completely.
- **How it changes:** This database is constantly being **modified**. When a new block arrives, your node reads the transactions, and for each one, it updates this state database. It might decrease one account's balance, increase another's, and change a contract's storage. It's a living, breathing database.

A node never blindly trusts its own local database. It constantly verifies its local database against the cryptographic commitments recorded on the blockchain.

- previousBlockHash: 00000000
- transactions: [Tx A]
- StateRoot
- Digital Signature of Validator
- Address of Validator
- Reward
- number

Rule of 2/3

Can a smart contract *start a transaction by itself*?

For example, could a Decentralized Finance (DeFi) lending protocol automatically send you your daily interest payment at exactly 8:00 AM every day, without anyone prompting it?

Smart Contract and a Token

Solidity Contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract SimpleCounter {
// State variable to store the counter value
uint256 public count;
// Constructor - runs once when contract is deployed
constructor() { count = 0;
// Initialize counter to 0
}
// Function to increment the counter by 1
function increment() public { count += 1; }
// Function to decrement the counter by 1
function decrement() public { require(count > 0, "Counter cannot go below zero"); count -= 1; }
// Function to get current count (already public, so this is optional)
function getCount() public view returns (uint256) { return count; }
// Function to reset counter to zero
```

```
function reset() public { count = 0; }
```

```
// Function to set counter to specific value
```

```
function setCount(uint256 _newCount) public { count = _newCount; } }
```

The Solution: The ERC-20 Token Standard

Instead of every developer inventing their own way to code a currency, the Ethereum community proposed a standard interface, a blueprint, called **ERC-20**

the contract has one primary variable in its storage: a

mapping

. You can think of this as a simple spreadsheet or a dictionary inside the contract itself.

mapping(address => uint256) public balances;

This line of code says: "Create a list where the key is an Ethereum address and the value is a number (the balance)."

- totalSupply(): A simple function that returns the total amount of the token that exists.
- balanceOf(address who): A function that looks up the balances mapping and returns the balance for the address who. (This answers Question 1).
- transfer(address to, uint256 amount): This is the key function. When you call it, the code does the following:
  Checks if your address (msg.sender) has enough tokens in the balances mapping.
  If yes, it subtracts the amount from your balance.
  It adds the amount to the to address's balance.
  If the check fails, it reverts the transaction.

ERC-20 is not a piece of software or a protocol. It is a technical *standard* or *specification*.

Think of it as a **blueprint** or a **universally agreed-upon set of rules** for creating a token on Ethereum.

- **Alice creates a token called "AliceCoin."** To check a balance, her function is named getAliceBalance(address). To send tokens, her function is sendAliceCoin(address, amount).
- **Bob creates "BobCoin."** His functions are fetch_balance(address) and move_tokens(address, amount).
- **Carol creates "CarolCoin."** Her functions are balanceOf(address) and do_transfer(address, amount).

The Solution: A Common Interface

The developers in the Ethereum community recognized this problem early on. A group of them wrote **E**thereum **R**equest for **C**omments **#20**. This document proposed a standard.

It said: "If you want to create a fungible token (where each token is identical, like a dollar bill), your smart contract **MUST** implement the following set of functions and events with these exact names and parameters."

ERC-20 is this mandatory public interface. It's a contract that a smart contract makes with the outside world.

Introduction To Solidity

Lesson 1: Dealing with simple variable

```
// 1. We tell the compiler what license we're using. It's just a good habit.
// SPDX-License-Identifier: MIT

// 2. We tell the compiler which version of Solidity to use.
pragma solidity ^0.8.20;

// 3. We define our contract. Think of it like a 'class' in C++.
contract HelloWorld {
    // 4. This is a "State Variable". It is data stored permanently
    //    with the contract on the blockchain.
```

```
    string public greeting = "Hello, World!";
}
```

Task:

- Create a new contract named MyFavoriteNumber.
- Inside this contract, create one state variable.
- The variable should be named myNumber.
- It should be a uint (unsigned integer, a positive number).
- It should be public so we can easily read it.
- Give it any initial value you want (e.g., uint public myNumber = 7;).

Lesson 2: Dealing with function

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyFavoriteNumber {
    uint public myNumber = 10;

    // This is our new function!
    function setMyNumber(uint _newNumber) public {
        myNumber = _newNumber;
    }
}
```

Task:

- Add a new public state variable to your contract. Make it a string and name it myStatus. Give it an initial value like "Learning Solidity".
- Add a new public function called setMyStatus.
- This function should take one string as input. just use the keyword calldata. So the input will look like string calldata _newStatus.
- The function should update the myStatus state variable with the new input.

Lesson 3: **Functions That Read and Return Data**

We also have special keywords to tell Solidity that a function will **not** change any data. This is important because it means calling the function is **free**.

The two keywords are:

- **view**: Use this for a function that only **reads** state variables but doesn't change them.
- **pure**: Use this for a function that doesn't even read the state variables. It's a pure calculation that only depends on its inputs.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyInfo { // I've renamed the contract to be more descriptive
    uint public myNumber = 10;
    string public myStatus = "Learning Solidity";

    function setMyNumber(uint _newNumber) public {
        myNumber = _newNumber;
    }

    function setMyStatus(string calldata _newStatus) public {
        myStatus = _newStatus;
    }

    // This is our new VIEW function!
    function getInfo() public view returns (string memory) {
        // NOTE: Combining strings in Solidity is a bit strange.
        // We use a special function called abi.encodePacked.
```

```
        // Don't worry about how it works for now.
        return string(abi.encodePacked("My number is ", toString(myNumber), " and I am currently ", myStatus));
    }

    // This is a PURE helper function. It doesn't read any state.
    // Its only job is to turn a number into a string.
    function toString(string memory _i) internal pure returns (string memory) {
        if (_i == 0) { return "0"; }
        uint j = _i;
        uint len;
        while (j != 0) { len++; j /= 10; }
        bytes memory bstr = new bytes(len);
        uint k = len;
        while (_i != 0) {
            k = k-1;
            uint8 temp = (48 + uint8(_i - _i / 10 * 10));
            bytes1 b1 = bytes1(temp);
            bstr[k] = b1;
            _i /= 10;
        }
        return string(bstr);
    }
}
```

Task:

- Add a new function to your contract called doubleMyNumber.
- It should take one uint as input (e.g., uint _numberToDouble).
- It should be public.
- It must be marked as pure, because it's just doing math and doesn't need to read myNumber or myStatus.
- It must return a uint.
- The logic inside the function should be simple: return _numberToDouble * 2;

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MathPractice {
    function sumOfEvenNumbers(uint _n) public pure returns (uint) {
        uint total = 0;
        for (uint i = 1; i <= _n; i++) {
            if (i % 2 == 0) {
                total = total + i;
            }
        }
        return total;
    }
}
```

Deep Dive on string and bytes

In many programming languages, strings are easy to work with. In Solidity, they are a bit more complex and less flexible due to the design of the EVM and the cost of gas. Understanding the difference between string and bytes is key to writing efficient code.

bytes: The Efficient Foundation

- **What it is:** A bytes variable is a dynamically-sized array of raw bytes. Think of it as byte[]. This is the EVM's "native" way of handling dynamic data.
- **Best for:** Raw, unstructured data that you don't necessarily need to read as human text. For example, a cryptographic signature, the bytecode of another contract, or any raw binary data.
- **Key Advantage:** bytes is more gas-efficient than string for on-chain operations. You have more direct control over it.

`You can get the length of a bytes array directly:`

```
bytes myData = "some data";
```

```
uint length = myData.length; // This works
```

```
You can also access individual bytes by index:
```

```
byte firstByte = myData[0]; // This works
```

string: The Human-Readable Layer

- **What it is:** A string variable is also a dynamically-sized byte array, but it's specifically encoded in **UTF-8**.
- **Best for:** Human-readable text that you expect to display on a website frontend. For example, a user's name, a status message, or an NFT's description.
- **Key Disadvantage:** Solidity does **not** provide many built-in tools to manipulate strings directly on-chain because it's very expensive.
  You **cannot** get the length of a string with .length directly (myString.length does not work).
  You **cannot** access characters by index directly (myString[0] does not work).
  You **cannot** concatenate (join) two strings easily with + ("a" + "b" does not work).

```
string myStatus = "Learning Solidity";

// To get the length of the string:
uint length = bytes(myStatus).length; // This returns 17

// To get the first character (as a byte):
bytes1 firstCharAsByte = bytes(myStatus)[0]; // This gives the byte for 'L'
```

```
string name = "Rohit";
string status = "is learning";

// This is how you join them:
string sentence = string(abi.encodePacked(name, " ", status, "."));
// The value of 'sentence' will be "Rohit is learning."
```

Lesson 4: **Adding Rules with require**

So far, anyone can call our functions and set any value they want. What if we want to add some rules? For example, what if we want to prevent someone from setting the myNumber to zero?

This is where require() comes in. It's one of the most important functions in Solidity.

require() checks if a condition is true.

- If the condition **is true**, the function continues.
- If the condition **is false**, the function stops immediately, reverses any changes it made, and gives back an error message.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyFavoriteNumber {
    uint public myNumber = 10;
    string public myStatus = "Learning Solidity";

    // Let's add the setMyNumber function back in for this example
    function setMyNumber(uint _newNumber) public {
        // This is our new rule!
        // It checks if the new number is greater than the current one.
        require(_newNumber > myNumber, "New number must be greater than the current one.");

        // This line will only run if the require() check passes.
        myNumber = _newNumber;
    }

    function setMyStatus(string calldata _myStatus) public {
        myStatus = _myStatus;
```

```
    }

    function doubleMyNumber(uint _num) public pure returns (uint) {
        return _num * 2;
    }
}
```

Task:

- Modify the setMyStatus function.
- Add a require() statement at the beginning of the function.
- The rule should be: **The new status string cannot be empty.**
- Give it a helpful error message, like "Status cannot be empty.".

Lesson 5: **msg.sender and Contract Ownership**

Every time someone calls a function on your contract, the Ethereum network provides some special information about the call. The most important piece of information is **who made the call**.

Solidity gives us a special, "magic" global variable for this: msg.sender.

- msg.sender: Is always equal to the **address** of the person or contract that is currently calling the function.

This allows us to create rules based on *who* is calling, not just *what* they're sending. This is how we implement **access control** or **ownership**.

The constructor: A Special Function

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// This contract has an owner!
contract Owned {
    // 1. A state variable to store the owner's address.
    address public owner;

    // 2. The constructor function.
    // This runs automatically ONLY when the contract is deployed.
    constructor() {
        // 3. We set the 'owner' to the address of the person who deployed it.
        owner = msg.sender;
    }

    // A function that can only be called by the owner.
    function doSomethingOnlyOwnerCanDo() public {
        // 4. We require that the person calling this function IS the owner.
        require(msg.sender == owner, "You are not the owner!");

        // ... owner-only logic would go here ...
    }
}
```

Task:

- Add a new public state variable of type address named owner.
- Add a constructor to your contract. Inside the constructor, set the owner variable to msg.sender.
- Modify the setMyNumber function. Add a **new require statement** at the very beginning of the function to check that msg.sender == owner. The error message should be something like "Only the owner can set the number.".

Lesson 6: Custom Modifiers

What if you have ten different functions that should only be callable by the owner? You would have to copy and paste that same require statement into all ten functions. This is repetitive and can lead to errors.

Solidity has a beautiful solution for this called a **modifier**.

A modifier is like a "wrapper" or a "pre-check" that you can attach to a function.

```
// 1. Define the modifier
modifier onlyOwner() {
    require(msg.sender == owner, "Only the owner can call this function.");
    // 2. This is a special placeholder. It means:
    // "If the require check passed, run the rest of the function's code now."
    _;
}
```

```
// Attach the modifier right after the visibility keyword
function setMyNumber(uint _num) public onlyOwner {
    // The ownership check is now handled automatically by the modifier!
    // We can remove the require line from inside the function.
    require(_num > myNumber, "Your number must be greater than the current number");
    myNumber = _num;
}
```

Task:

- In your MyFavoriteNumber contract, create a new modifier named onlyOwner.
- The modifier should contain the require statement that checks if msg.sender == owner.
- Remove the require statement from inside the setMyNumber function and add the onlyOwner modifier to its declaration instead.
- For extra practice, let's say that anyone can *read* the status, but only the owner should be able to *change* it. Add the onlyOwner modifier to the setMyStatus function as well.

Lesson 7: Ether, Units, and payable Functions

Smart contracts can hold, receive, and send Ether, the native currency of the Ethereum blockchain. This is what makes them so powerful for finance, NFTs, and more.

Ether Units

First, it's important to know that while we talk about "ether," smart contracts do all their calculations in the smallest unit, called **wei**.

It's like how we have dollars and cents. You wouldn't do financial calculations in a mix of dollars and cents; you'd convert everything to cents first.

Here's the breakdown:

- 1 ether = 1,000,000,000,000,000,000 wei (1 with 18 zeros)
- 1 gwei = 1,000,000,000 wei (1 with 9 zeros) - *Gwei is often used to talk about gas prices.*

Solidity gives us convenient keywords for these units, so you don't have to type all the zeros.

- 1 ether
- 1 gwei
- 1 wei

So, 2 ether is the same as 2000000000000000000 wei. The keywords just make it readable.

Receiving Ether: The payable Keyword

By default, functions in a smart contract **cannot** accept Ether. If you try to send Ether to a normal function, the transaction will be rejected. This is a safety feature.

To make a function able to receive Ether, you must mark it as payable.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract TipJar {
    // We can see how much Ether is stored in this contract
```

```
    function getBalance() public view returns (uint) {
        // 'address(this).balance' is a special property that
        // gives the Ether balance of the current contract.
        return address(this).balance;
    }

    // This is a payable function. Anyone can call it and send Ether.
    function sendTip() public payable {
        // The function body can be empty! Its only job is to
        // receive the Ether.
    }
}
```

Task

- Modify your setMyNumber function.
- Make the function payable.
- Add a **new rule** (require statement). This rule should check that the amount of Ether sent with the function call is **exactly 0.01 ether**.

  Hint 1: Just like msg.sender, Solidity gives us another magic variable, msg.value, which holds the amount of wei sent with the call.

  Hint 2: You can use the ether keyword for this. The condition will look like: msg.value == 0.01 ether.
- Give it a good error message, like "You must pay exactly 0.01 ETH.".